Parallel Algorithms

- Parallel computer
  - A computer with more than one processing unit
    multiprocessor systems
    multicore systems
    distributed systems
    …
- Parallel algorithm
  - An algorithm developed for a parallel computer
    with special instructions

Parallel Algorithms

- Speed
  - Example

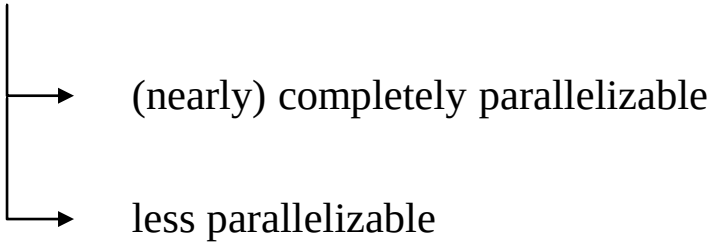
    Comparison-based sorting

    Sequential $\rightarrow \theta(n \log n)$

    Parallel $\rightarrow \theta(n)$

Parallel Algorithms – Problem types

- How much can we gain?

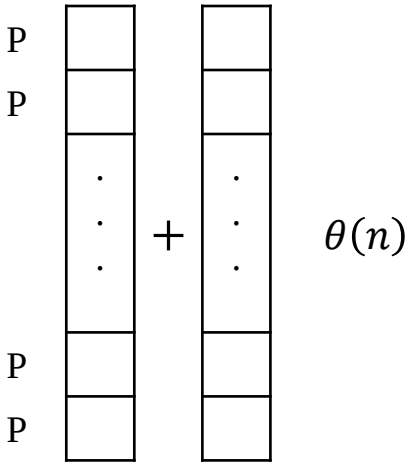


Parallel Algorithms – Problem types

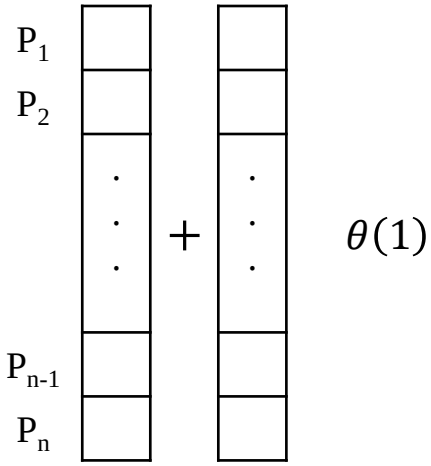
- How much can we gain?
  - (nearly) completely parallelizable
  - less parallelizable

- Example: Vector addition $c_i = a_i + b_i \ , \ \forall i$

Sequential



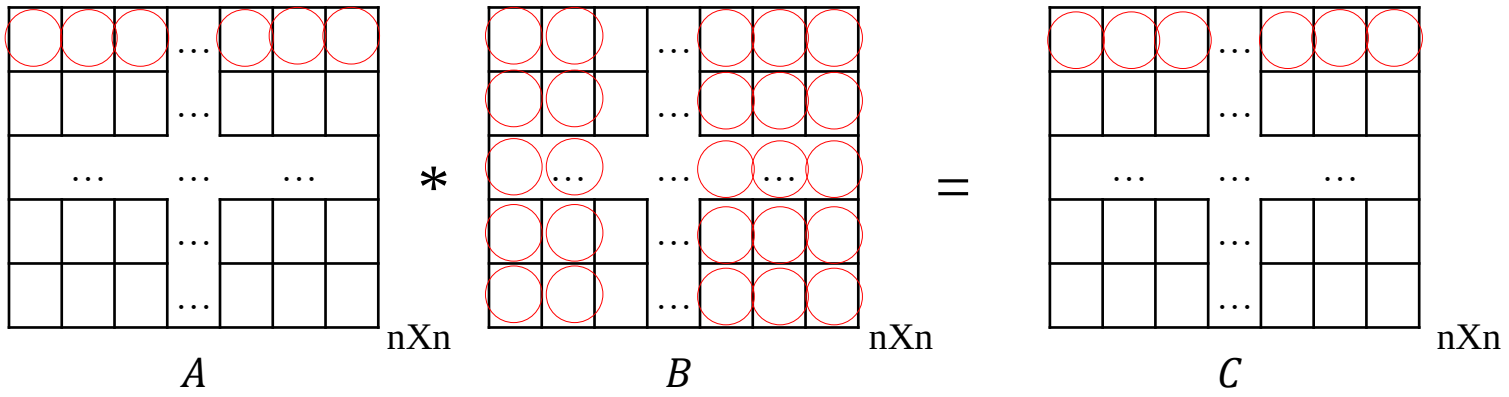
Parallel



Parallel Algorithms – Problem types

- How much can we gain?
  - (nearly) completely parallelizable
  - less parallelizable
- Example: Matrix multiplication

Parallel



Parallel Algorithms – Problem types

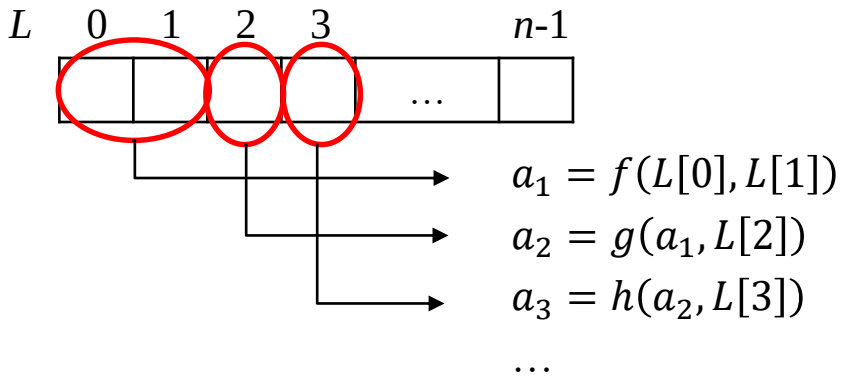
- How much can we gain?

(nearly) completely parallelizable

less parallelizable

  - Example: An extreme example

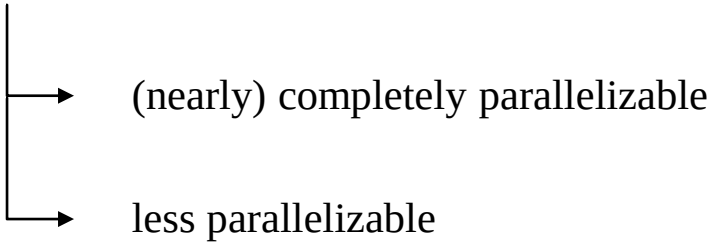
Parallel



sequential complexity
=
parallel complexity

Parallel Algorithms – Problem types

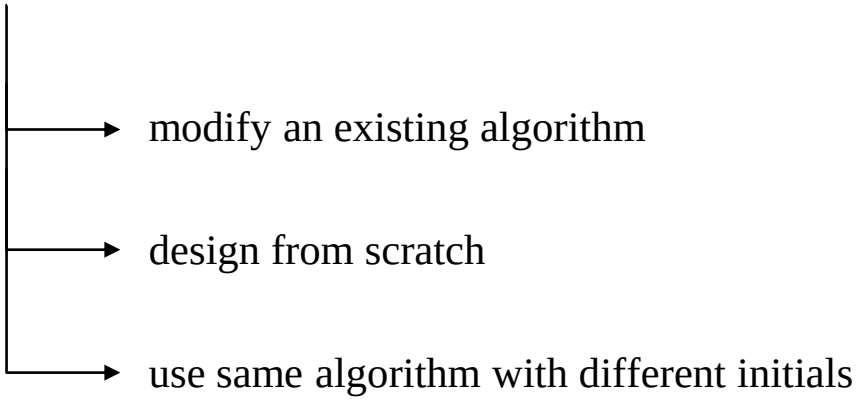
- How much can we gain?



develop a parallel algorithm for a problem only if you will get a significant performance gain over the sequential solution

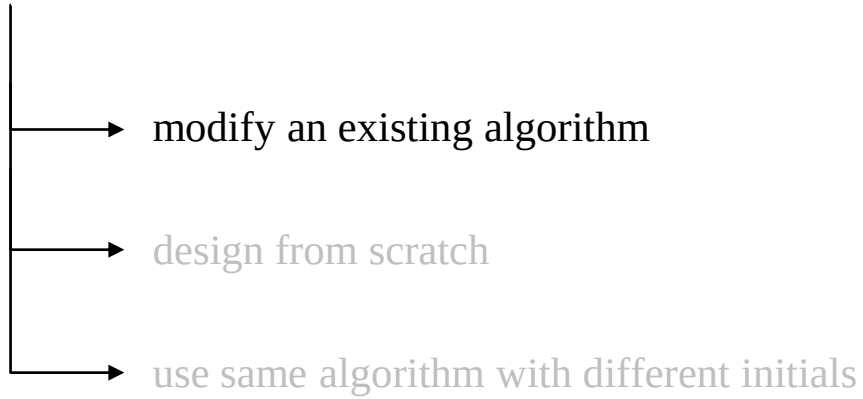
Parallel Algorithms – Approaches

- How to design?



Parallel Algorithms – Approaches

- How to design?

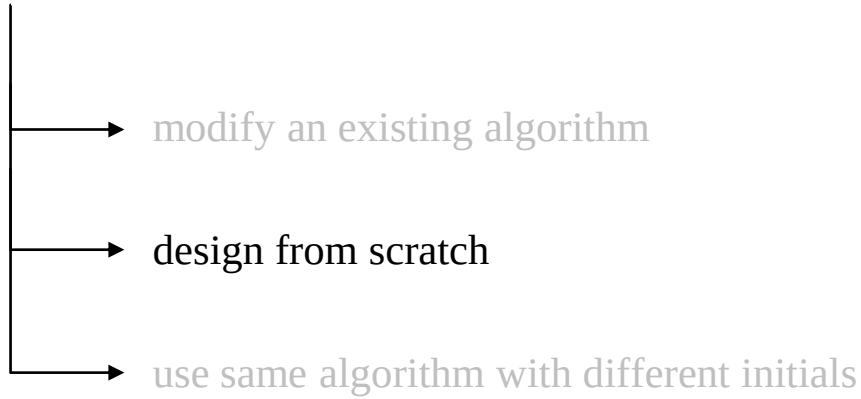


  - Change the code

    Example: parallel (linear) search

Parallel Algorithms – Approaches

- How to design?

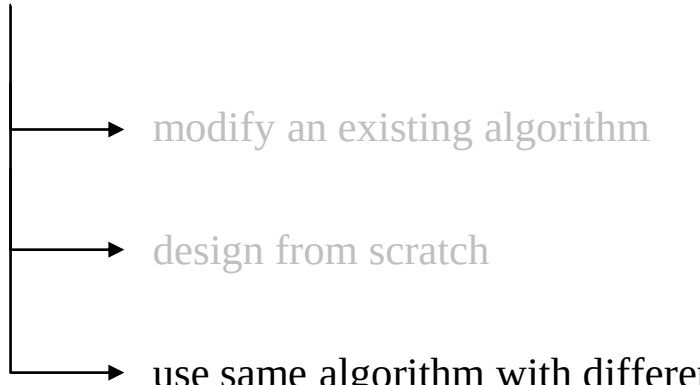


  - A new way of solving the problem

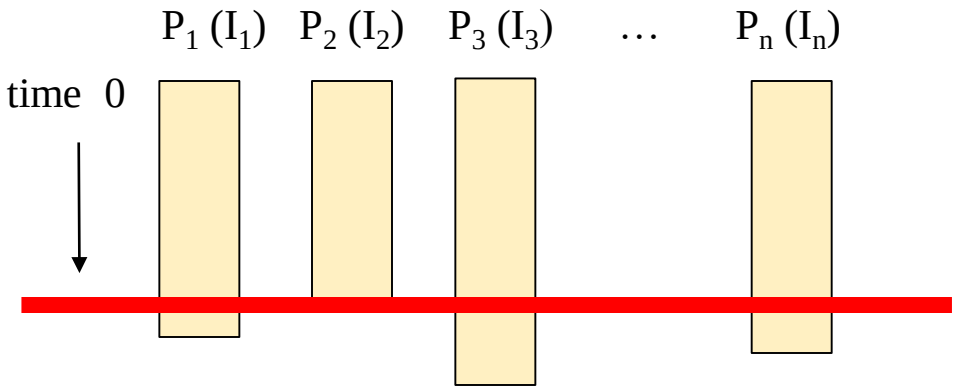
    Example: sorting in snake order

Parallel Algorithms – Approaches

- How to design?



  - Example: root finding e.g. $f(x) = e^{2x} - x - 6$



Example:
Matrix multiplication ???

Parallel Algorithms – Architecture

- Sequential computer
  - 1 processor
  - processor ↔ memory communication
- Parallel computer
  - number of processors?
  - processor ↔ processor communication?
  - processor ↔ memory communication?

Parallel Algorithms – Architecture

- We need to know
  - how many processors exist
  - powerful or simple processors
    - Example: Searching a list of size n ($n \leq 1000$)

      10 processors
      1000 processors
      } different algorithms
    - Example: Root finding

      All powerful processors

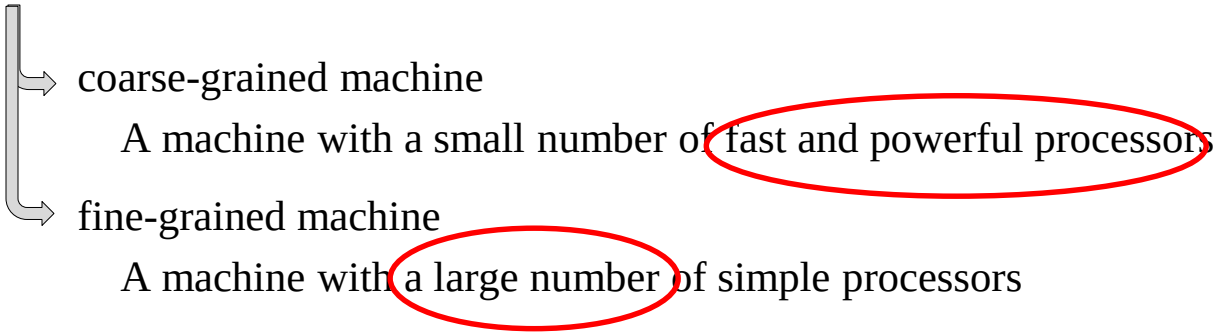
      Dummy processors → complex operations with a central processor

Parallel Algorithms – Architecture

- Granularity
  - The number and power (type) of processors in a machine
    - coarse-grained machine
      - A machine with a small number of fast and powerful processors
    - fine-grained machine
      - A machine with a large number of simple processors
  - Number of processors
    - a function of data size – $p(n)$ → Not a realistic assumption
    - fixed – p

Parallel Algorithms – Architecture

- Granularity
  - The number and power (type) of processors in a machine

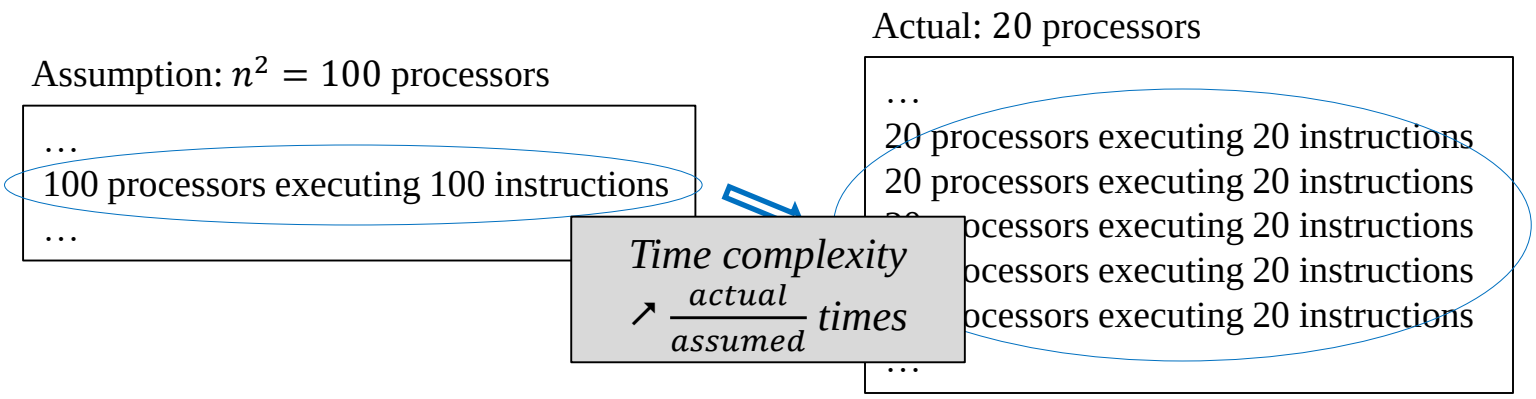


  - Don't deal with arranging the design wrt different number of processors

    Deal with the logic behind the algorithm

Parallel Algorithms – Architecture

- Granularity
  - The number and power (type) of processors in a machine
    - coarse-grained machine
      - A machine with a small number of fast and powerful processors
    - fine-grained machine
      - A machine with a large number of simple processors
  - Division of labor

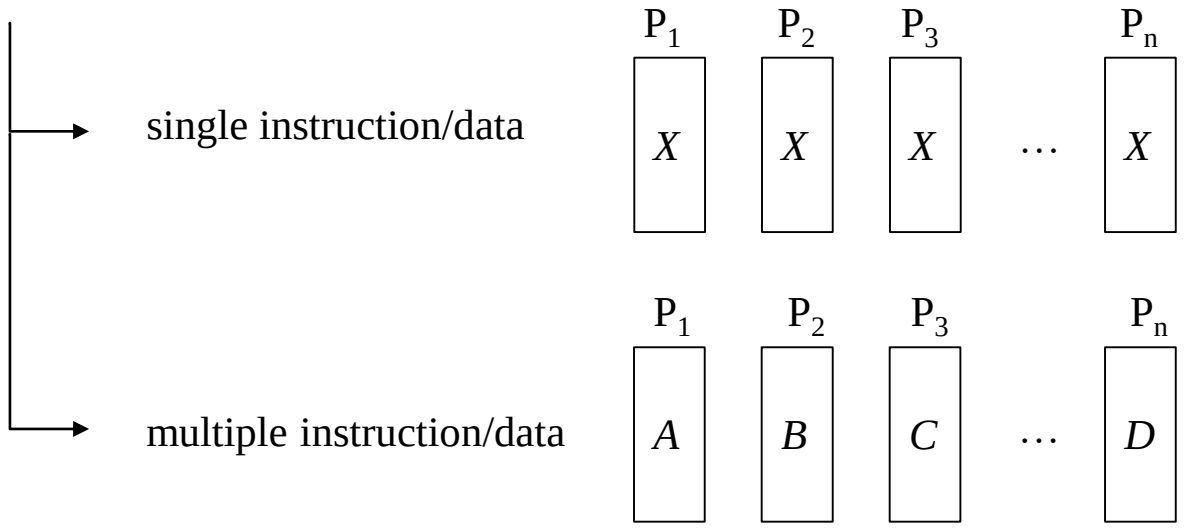


Parallel Algorithms – Flynn's Taxonomy

- A classification of computer architectures based on the concepts of instruction stream and data stream

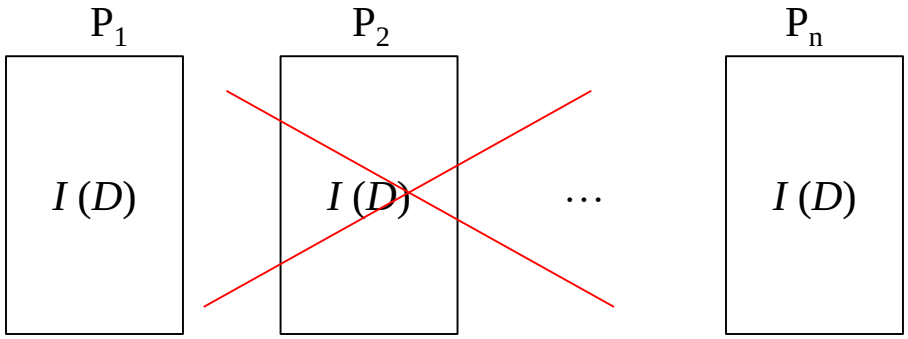
a sequence of instructions executed by the computer

a sequence of data items operated on by the instruction stream

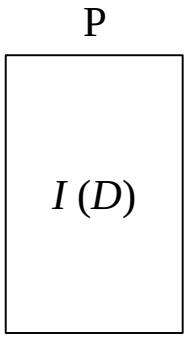


Parallel Algorithms – Flynn's Taxonomy

- SISD (single instruction, single data)

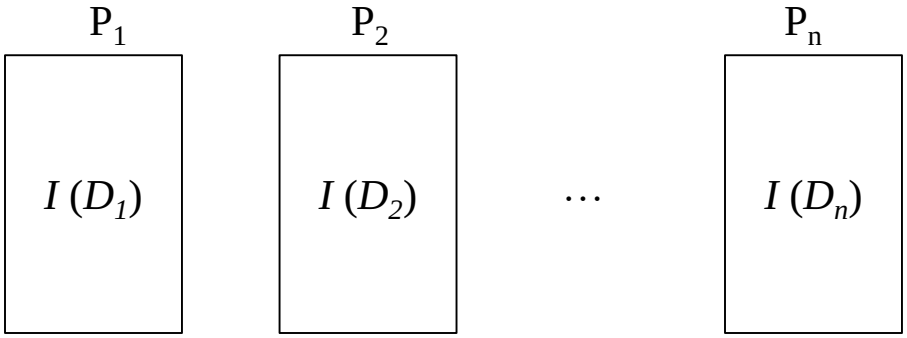


  - Makes sense?



Parallel Algorithms – Flynn's Taxonomy

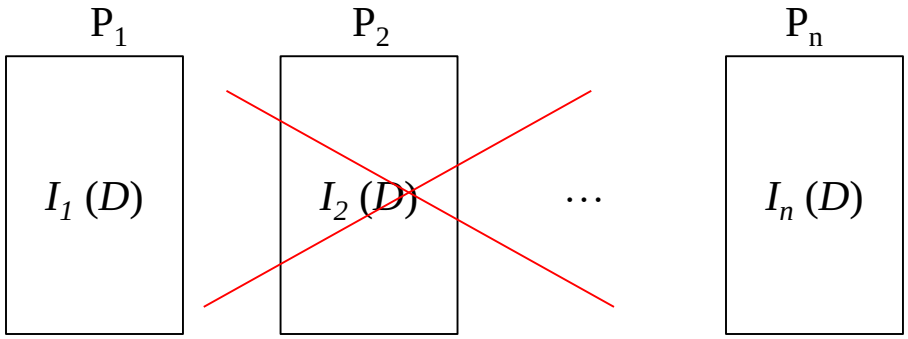
- SIMD (single instruction, multiple data)



  - Central processor (front-end processor)
    - Stores the program, broadcasts one instruction per clock cycle
    - Processors operate in synchronized manner
  - Each processor executes the instruction on data in local memory
  - Each processor is active or idle
    - Through the use of a mask

Parallel Algorithms – Flynn's Taxonomy

- MISD (multiple instruction, single data)



  - Makes sense?

  - Example: Primality checking

    A hypothetical machine — Problem: *n* is prime?

    Instruction set

    Divide by 2 — P_1 : Divide *n* by 2

    Divide by 3 — P_2 : Divide *n* by 3

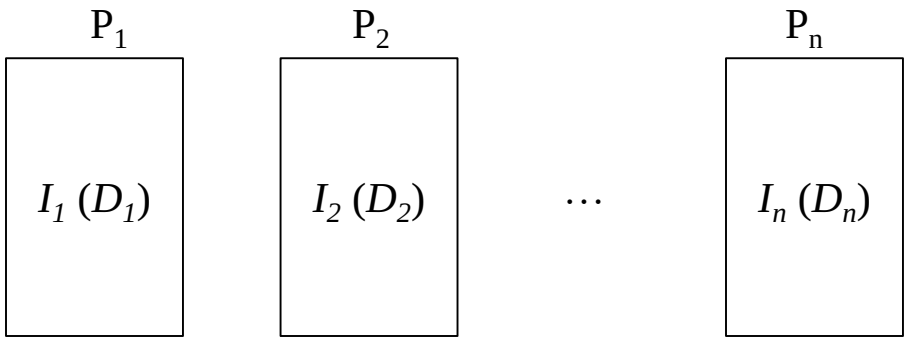
    … — …

    $P_{\sqrt{n}-1}$: Divide *n* by $\sqrt{n}$

    $\theta(1)$

Parallel Algorithms – Flynn's Taxonomy

- MIMD (multiple instruction, multiple data)



  - Each processor stores and executes its own program
  - Synchronized at some points



SPMD (single program, multiple data) programming style

single program

each processor operating on different data

Parallel Algorithms – Flynn's Taxonomy

- Models
  - SISD → sequential computer
  - SIMD ⇒ this course
  - MISD → does not make sense
  - MIMD ⇒ MPI project
    - processors communicate by sending/receiving messages
    - no central processor (distributed computing)
    - Appendix F (textbook) covers MPI language

Parallel Algorithms – Communication

- Issues
  - Number of processors ✔
  - Types of processors ✔
  - Single/Multiple instruction/data ✔
  - Processor–Memory connection
  - Processor–Processor communication

Parallel Algorithms – Communication

- Tightly coupled system
  - Centralized memory
  - Processors are only connected to memory
  - Communication is via memory

PRAM

- Loosely coupled system
  - No centralized memory
  - Processors have own memory
  - Processors are connected to each other
  - Communication is via connections

Interconnection network